University of BRISTOL

Cyber Security Assignment

File Integrity Checker
based on OpenSSL in C

MSc Foundations of Cyber Security

Course Code: COMSM0118_2025_TB-1

Individual Assessment (60%)

Name: Yi Li
Student ID: 2829967
Email: to25233@bristol.ac.uk
Facutly: Science and Engineering

17th October 2025

Contents

1 Introduction

In today's environment of growing cyber threats, data integrity has become increasingly critical. Files may be subject to malicious tampering or accidental corruption during distribution, transmission, or long-term storage. Any unauthorized or undetected modifications can lead to severe consequences, including system failures, data breaches, and significant financial losses [1, 2]. Therefore, establishing an efficient and reliable file integrity verification mechanism is essential for ensuring system security and data trustworthiness. This project aims to develop a C-language command-line tool that utilizes the OpenSSL cryptographic library to invoke SHA-256 and SHA-512 hash algorithms. This enables encryption and integrity verification for files of any size. This article provides a detailed exposition of the tool's design philosophy, core program implementation, and test results, among other key report contents.

2 Literature Review and Critical Analysis

2.1 Cryptographic Hash Functions

A hash algorithm is a method for creating small numerical "fingerprints" from any file. Like fingerprints, hash algorithms serve as a unique identifier for files using a shorter piece of information. This identifier correlates with every byte of the file and is difficult to reverse. Specifically, when the original file changes, its identifier value also changes, alerting users that the current file is no longer the one they intended [2, 3].

Therefore, it has the following characteristics:

- **Determinism:** The same input always produces the same hash output
- **Unidirectionality:** It is computationally infeasible to deduce the original input data from the hash value.
- **Collision Resistance:** It is computationally difficult to find two different input data that produce the same hash value.
- **Avalanche Effect:** Small changes in input can produce significant changes in output

In addition, there are many common hash algorithms, including MD5, SHA-1, SHA-2 family and SHA-3. The choice of cryptographic hashing algorithm is crucial to the effectiveness and security of any file integrity verification system.

2.2 Justification of Algorithm Selection

This project adhered to current industry best practices and security standards in selecting cryptographic hash algorithms, ultimately choosing SHA-256 (default) and SHA-512. This decision was based on a comprehensive consideration of algorithm security, performance, and standards compliance.

SHA-256 generates a 256-bit hash value (32 bytes, 64 hexadecimal characters) and provides a collision resistance level of 2^{128}, sufficient to meet the security requirements of most

applications. SHA-512 generates a 512-bit hash value (64 bytes, 128 hexadecimal characters) and provides a higher security margin and is particularly suitable for scenarios with extremely high long-term security and data confidentiality requirements [4]. Thus, they are currently the most widely accepted and recommended family of hash algorithms [5–7]. Also, the SHA-2 algorithm is part of the FIPS standard and is recognized by both the government and industry. At the implementation level, the OpenSSL library provides highly optimized and stable low-level API support for SHA-256 and SHA-512, ensuring performance and reliability across various platforms [8].

In contrast, older algorithms such as MD5 have been shown to suffer from severe collision vulnerabilities, while SHA-1 has been deprecated due to the risks of both theoretical and practical attacks. The choice of SHA-256 and SHA-512 ensures that the hash values have sufficient entropy and collision resistance, effectively protecting against birthday attacks [4].

2.3 Analysis of Design Trade-offs

2.3.1 Streaming I/O vs. Memory-Mapped I/O

This tool adopts a streaming file processing model, looping through a fixed-size 4KB buffer to read file data blocks and using OpenSSL function SHA_Update function for incremental hash calculation.This approach ensures that the program's memory usage is constant, regardless of the size of the file being processed. It avoids loading the entire file into memory at once, preventing memory overflow or system resource exhaustion.

2.3.2 Hash Value Verification

The program allows case-insensitive input, which improves user-friendliness because users may copy hashes from different sources, which may use uppercase or lowercase letters. Cryptographically, the case of the hexadecimal representation of a hash value does not affect its security, so this trade-off improves usability without sacrificing security.

2.3.3 API Low Layer vs. EVP Interface

Due to course requirements, a low-level interface was selected to implement the hash encryption verification function. For tasks that only need to support two specific algorithms, SHA-256 and SHA-512, the implementation logic of the low-level API is more direct, requires less code, and is easy to understand and debug.

3 Design, Implementation and Security Evaluation

3.1 Module Division and Workflow

The program's architecture is divided into three core functional modules, corresponding to the function groupings in the code, which helps improve code readability and maintainability:

- **Core logic module:** It is responsible for program startup, argument parsing (parse_arguments) and process control (run_hash_process). This is the coordination center for user interaction and internal system operations.
- **Hashing module:** It involves cryptographic operations and is responsible for algorithm verification (is_valid_algorithm), file streaming (compute_file_hash), and final hash comparison (verify_hash).
- **Utility Module:** It is responsible for providing user interface elements such as help information (print_usage) and a list of supported algorithms (print_supported_algorithms).

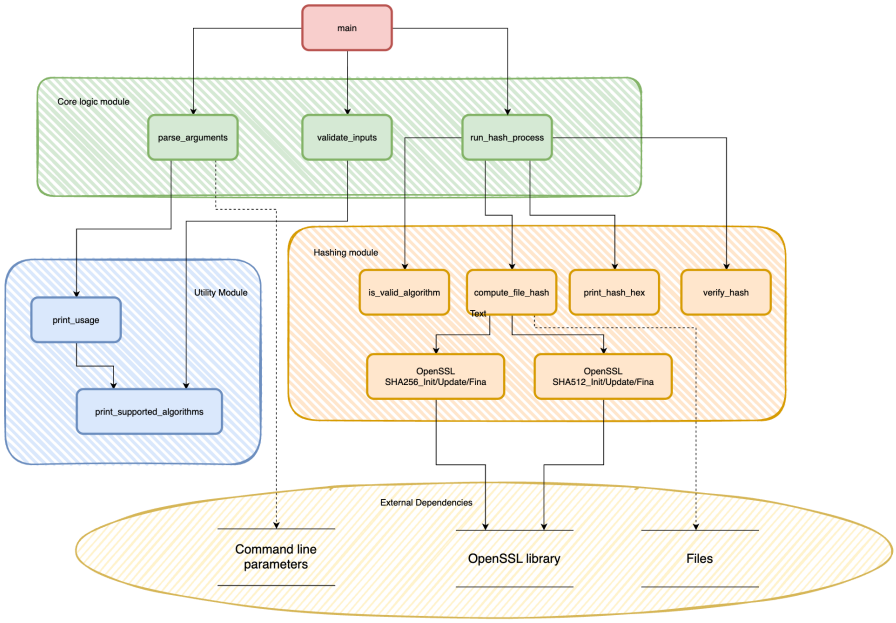

Figure 1: Program workflow of the file integrity checker

As shown in Figure1, the program starts with the main function, which first calls parse_arguments to parse the command-line arguments to determine the algorithm, file name, and expected hash value. It then verifies the legitimacy of the input using validate_inputs and is_valid_algorithm. Once the input is verified, run_hash_process coordinates the core process, calling compute_file_hash to calculate the file hash value in a streaming manner and outputting the result using print_hash_hex. If the user provides an expected hash, verify_hash is called for comparison and verification. Finally, a success or failure message is output based on the verification result. A comprehensive error handling mechanism is used throughout the process to ensure the program exits safely.

3.2 Key Implementation Details

In this project, the compute_file_hash and verify_hash functions serve as the core functional components of the program. The compute_file_hash function handles the computation, utilizing a streaming file processing architecture based on a fixed 4KB buffer. This ensures consistent memory usage regardless of file size, thereby preventing memory exhaustion risks associated

with processing extremely large files. The function directly invokes underlying APIs provided by OpenSSL, including the SHA256_Init/Update/Final and SHA512_Init/Update/Final series of functions. This approach guarantees computational efficiency while providing a unified processing workflow for different algorithms.

On the other hand, the verify_hash function handles integrity verification. This function first converts the computed binary hash result into a standard hexadecimal string representation, enabling more intuitive comparison against the expected hash value. Furthermore, it employs the ‘strcasecmp‘ function for case-insensitive string comparison. However, this conventional string comparison approach carries potential risks of timing attacks, where attackers could infer partial hash information by precisely measuring the execution time of comparison operations.

3.3 Test Result

According to Table 1, SHA256 is approximately 25% faster when processing small 1KB files. However, as file sizes increase, SHA512 demonstrates superior performance, achieving a 25-33% performance boost for files of 1MB and larger. This validates SHA512's optimization for large file processing on modern 64-bit architectures, while also demonstrating SHA256's practicality as the default algorithm for small file processing and compatibility.

Table 1: SHA-256 vs SHA-512 on time (sec)

size	**SHA256**	**SHA512**
1KB	0.0003	0.0004
1MB	0.0061	0.0046
100MB	0.3118	0.2081
1GB	2.9928	2.0554

The code generating the test data will be put in appendix session for checking.

4 Conclusion

This project successfully developed the filehash file integrity verification tool based on OpenSSL, implementing both the SHA-256 and SHA-512 algorithms. Performance testing showed that SHA-256 outperformed small files, while SHA-512 offered significant performance improvements with large files. The tool utilizes streaming processing to ensure robustness. While currently supported algorithms are limited and lack parallel optimization, it lays the foundation for future integration with enhanced features such as SHA-3 and parallel computing.

References

[1] Verizon Business, “2024 data breach investigations report,” tech. rep., Verizon, 2024.

[2] D. Craigen, N. Diakun-Thibault, and R. Purse, “Defining cybersecurity,” *Technology innovation management review*, vol. 4, no. 10, 2014.

[3] J. H. Saltzer and M. D. Schroeder, “The protection of information in computer systems,” *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

[4] National Institute of Standards and Technology, “Secure hash standard (SHS),” FIPS PUB 180-4, U.S. Department of Commerce, August 2015. Includes updates through March 2022.

[5] A. K. Lenstra and E. R. Verheul, “Selecting cryptographic key sizes,” *Journal of cryptology*, vol. 14, no. 4, pp. 255–293, 2001.

[6] D. R. Stinson, *Cryptography: theory and practice*. Chapman and Hall/CRC, 2005.

[7] R. Anderson, *Security engineering: a guide to building dependable distributed systems*. John Wiley & Sons, 2010.

[8] J. Viega, M. Messier, and P. Chandra, *Network security with openSSL: cryptography for secure communications*. " O’Reilly Media, Inc.", 2002.

Declaration

I declare that in this project, AI was used as an auxiliary to help me learn C language, algorithm architecture and article logic. The final submission represents my own work and understanding of the topic.

A Appendix

Listing 1: code for generating samples

```

import os
import random
def generate_random_file(filename, size):
    """Generate a random text file of a specified size"""
    # 1MB = 1024*1024 bytes
    sizes = {
        '1KB': 1024,
        '1MB': 1024*1024,
        '100MB': 100*1024*1024,
        '1GB': 1024*1024*1024
    }

    target_size = sizes[size]
    with open(filename, 'w') as f:
        chunk_size = 1024
        written = 0
        while written < target_size:
            remaining = target_size - written
            current_chunk = min(chunk_size, remaining)

            # Generate random printable characters
            random_chars = ''.join(chr(random.randint(32, 126)) for _ in
                range(current_chunk))
            f.write(random_chars)
            written += current_chunk

    print(f"            : {filename} ({size}),             : {os.path.
       getsize(filename)} bytes")

# Generate four test files
generate_random_file("test_1KB.txt", "1KB")
generate_random_file("test_1MB.txt", "1MB")
generate_random_file("test_100MB.txt", "100MB")
generate_random_file("test_1GB.txt", "1GB")
```